

MuJoCo Fluid Force 공식식과 v3.0 매핑 정리

MuJoCo 공식 문서 기준으로 각 항의 물리적 의미와 현재 UUV v3.0 구현 대응관계를 정리한 별도 문서

조백규 교수님 연구실 최애의 제자 박사과정 보고용 정리

2026-04-06

Contents

1 문서 목적과 기준	3
2 MuJoCo fluid model의 큰 구조	3
2.1 현재 v3.0 current path에서 active한 모델	3
3 Inertia model 공식식	4
3.1 Quadratic drag	4
3.2 Viscous resistance	4
4 Ellipsoid model 공식식	5
4.1 총합 구조	5
4.2 Projected area	5
4.3 Added mass	5
4.4 Quadratic translational drag	6
4.5 Angular drag	6
4.6 Linear viscous resistance	6
4.7 Magnus lift	7
4.8 Kutta lift	7
5 XML level에서 무엇을 바꾸면 어떤 항이 바뀌는가	8
6 현재 v3.0 current scene의 실제 값	8
7 current profile에서 MuJoCo fluid와 함께 바뀌는 값	8
8 MuJoCo 공식 항과 현재 v3.0 코드의 대응관계	9
8.1 무엇을 MuJoCo에 맡기고 무엇을 runtime이 보정하는가	10
9 Legacy와 current의 핵심 차이	10
9.1 Legacy는 무엇이였는가	10
9.2 Current는 무엇으로 바뀌었는가	10
9.3 안정화에 가장 크게 기여한 것은 무엇인가	10

10	각 항을 튜닝하면 어떤 현상이 바뀌는가	11
11	이 모델의 한계	11
12	결론	11

1 문서 목적과 기준

이 문서는 **MuJoCo 공식 문서의 fluid force 정의**를 기준으로, 현재 uuv_mujoco/v2.2 의 **v3.0 active current model** 이 그 공식을 어디까지 그대로 쓰고, 어디부터는 runtime에서 별도로 보정하는지를 정리한 별도 보고서다.

이 문서가 답하려는 질문은 네 가지다.

1. MuJoCo의 **inertia-based fluid model** 과 **ellipsoid-based fluid model** 은 각각 어떤 물리식으로 구성되는가.
2. ellipsoid 모델의 각 항 $\mathbf{f}_A, \mathbf{f}_D, \mathbf{f}_M, \mathbf{f}_K, \mathbf{f}_V, \mathbf{g}_A, \mathbf{g}_D, \mathbf{g}_V$ 는 무슨 의미를 가지는가.
3. XML의 density, viscosity, fluidshape, fluidcoef 가 실제로 어떤 항을 바꾸는가.
4. 현재 v3.0 current runtime에서 MuJoCo built-in fluid term과 Python runtime 보정 항이 어디서 갈리는가.

공식 기준 문서는 다음 두 페이지다.

- MuJoCo official documentation, [Fluid forces](#)
- MuJoCo official documentation, [XML Reference](#)

즉 이 문서의 공식식은 임의의 블로그나 2차 자료가 아니라 **MuJoCo 공식 문서에 나온 정의와 식**을 기준으로 다시 정리한 것이다. 여기서 **현재 repository 구현 설명**은 그 위에 얹은 코드 매핑 해설이다.

2 MuJoCo fluid model의 큰 구조

MuJoCo는 물속 유동장을 풀지 않는다. 대신 공식 문서가 명시하듯이 **state-less phenomenological model** 을 제공한다. 즉 주변 유체에 별도 상태 변수를 두지 않고, body 속도와 형상 근사만으로 수중 힘과 토크를 근사한다.

공식 문서 기준으로 fluid model은 두 종류다.

1. **Inertia model**: body 관성과 질량에서 equivalent inertia box를 만들고, 그 박스를 기준으로 drag와 viscous resistance를 계산한다.
2. **Ellipsoid model**: 각 geom을 ellipsoid로 근사하고, added mass, drag, Magnus lift, Kutta lift, viscous resistance까지 포함한 더 세분화된 fluid force를 계산한다.

두 모델 모두 medium의 **density** ρ 와 **kinematic viscosity** β 를 사용한다. 다만 ellipsoid model은 여기에 더해 **geom별 fluidcoef 다섯 개**를 받는다.

2.1 현재 v3.0 current path에서 active한 모델

현재 active current path는 tank_current_scene.xml 에 있는 fluid_* geom에 fluidshape="ellipsoid" 와 fluidcoef 를 정의하고, run_urdf_full.py 에서 **ellipsoid-only dynamics** 를 선택하는 구조다. 이 상태에서는:

- MuJoCo built-in ellipsoid fluid가 active이다.
- 공식 문서가 말하는 **body inertia sizes 기반 inertia model**은 parent body에서 비활성화된다.
- Python runtime의 예전 coefficient-based 6-DOF custom hydrodynamics는 current path에서 끈다.
- 대신 **buoyancy, distributed mass/inertia synthesis, thruster shaping, reaction torque, sensor export** 는 여전히 Python runtime이 담당한다.

즉 **현재 모델은 “순수 MuJoCo만”도 아니고 “순수 Python만”도 아니다**. fluid term은 MuJoCo ellipsoid model을 주력으로 쓰고, hydrostatic/actuation/interface는 runtime이 보강하는 하이브리드 구조다.

3 Inertia model 공식식

공식 문서에 따르면 inertia model은 body 질량 $\mathcal{M}$ 과 관성행렬 $\mathcal{I}$ 로부터 equivalent inertia box 반치수 (r_x, r_y, r_z) 를 만든다.

$$\begin{aligned} r_x &= \sqrt{\frac{3}{2\mathcal{M}} (\mathcal{I}_{yy} + \mathcal{I}_{zz} - \mathcal{I}_{xx})} \\ r_y &= \sqrt{\frac{3}{2\mathcal{M}} (\mathcal{I}_{zz} + \mathcal{I}_{xx} - \mathcal{I}_{yy})} \\ r_z &= \sqrt{\frac{3}{2\mathcal{M}} (\mathcal{I}_{xx} + \mathcal{I}_{yy} - \mathcal{I}_{zz})} \end{aligned}$$

그 다음 body frame 선속도 $\mathbf{v}$ 와 각속도 $\boldsymbol{\omega}$ 를 이용해

$$\begin{aligned} \mathbf{f}_{\text{inertia}} &= \mathbf{f}_D + \mathbf{f}_V \\ \mathbf{g}_{\text{inertia}} &= \mathbf{g}_D + \mathbf{g}_V \end{aligned}$$

를 계산한다. 여기서 D 는 quadratic drag, V 는 viscous resistance다.

3.1 Quadratic drag

$$\begin{aligned} f_{D,i} &= -2\rho r_j r_k |v_i| v_i \\ g_{D,i} &= -\frac{1}{2} \rho r_i (r_j^4 + r_k^4) |\omega_i| \omega_i \end{aligned}$$

의미는 단순하다.

- 선속도 크기가 커질수록 힘이 $|v|v$ 로 커진다.
- body가 어떤 축으로 넓게 보이느냐가 $r_j r_k$ 항에 들어간다.
- 회전 저항은 회전축 주위 swept area 효과가 $(r_j^4 + r_k^4)$ 꼴로 들어가서, 긴 body는 yaw/roll/pitch damping이 크게 달라질 수 있다.

3.2 Viscous resistance

$$\begin{aligned} f_{V,i} &= -6\beta\pi r_{eq} v_i \\ g_{V,i} &= -8\beta\pi r_{eq}^3 \omega_i \end{aligned}$$

여기서 $r_{eq} = (r_x + r_y + r_z)/3$ 이다. 이 항은 **저 Reynolds 영역의 선형 저항**에 해당한다.

영향:

- density와 달리 viscosity만 올려도 선형 damping이 증가한다.
- 속도가 아주 작을 때 잔류 진동이나 꼬리 흔들림을 정리하는 데 효과가 크다.
- 현재 프로젝트에서 “조종감이 너무 미끄럽다”를 잡을 때 viscosity나 선형 damping 계열이 중요한 이유가 이 항 때문이다.

4 Ellipsoid model 공식식

4.1 총합 구조

공식 문서 기준 ellipsoid model은 다음처럼 분해된다.

$$\begin{aligned}\mathbf{f}_{\text{ellipsoid}} &= \mathbf{f}_A + \mathbf{f}_D + \mathbf{f}_M + \mathbf{f}_K + \mathbf{f}_V \\ \mathbf{g}_{\text{ellipsoid}} &= \mathbf{g}_A + \mathbf{g}_D + \mathbf{g}_V\end{aligned}$$

항	물리적 의미	현재 tuning에서 체감되는 효과
$\mathbf{f}_A, \mathbf{g}_A$	added mass / added inertia	급가속, 급회전에서 “물까지 같이 끌고 가는” 무거운 느낌
$\mathbf{f}_D, \mathbf{g}_D$	quadratic drag / angular drag	속도가 커질수록 강하게 붙는 감쇠, 회전 꼬리 억제
$\mathbf{f}_M$	Magnus lift	회전하면서 전진할 때 옆으로 휘는 coupling
$\mathbf{f}_K$	Kutta lift	받음각이 생길 때 양력성 축력 발생
$\mathbf{f}_V, \mathbf{g}_V$	linear viscous resistance	저속 잔류 motion 정리, 점성 damping

4.2 Projected area

ellipsoid drag와 lift 식에는 projected area가 반복해서 들어간다. 공식 문서가 제시하는 projected area는

$$A_{\mathbf{u}}^{\text{proj}} = \pi \sqrt{\frac{r_y^4 r_z^4 u_x^2 + r_z^4 r_x^4 u_y^2 + r_x^4 r_y^4 u_z^2}{r_y^2 r_z^2 u_x^2 + r_z^2 r_x^2 u_y^2 + r_x^2 r_y^2 u_z^2}}$$

이다. 여기서 $\mathbf{u}$ 는 속도 방향 단위벡터다.

의미:

- 속도 방향이 바뀌면 projected area도 바뀐다.
- 같은 ellipsoid라도 broadside로 움직일 때와 slender 방향으로 움직일 때 drag가 달라진다.
- 결국 **geom size**와 **geom orientation** 이 fluid force 방향성과 크기를 동시에 결정한다.

4.3 Added mass

공식 문서는 added mass를 potential flow 관점으로 설명하고, 세 대칭면을 가진 body에 대해 다음 compact form을 제시한다.

$$\begin{aligned}\mathbf{f}_A &= -\mathbf{m}_A \circ \dot{\mathbf{v}} + (\mathbf{m}_A \circ \mathbf{v}) \times \boldsymbol{\omega} \\ \mathbf{g}_A &= -\mathbf{l}_A \circ \dot{\boldsymbol{\omega}} + (\mathbf{m}_A \circ \mathbf{v}) \times \mathbf{v} + (\mathbf{l}_A \circ \boldsymbol{\omega}) \times \boldsymbol{\omega}\end{aligned}$$

즉 added mass는 단순 damping이 아니라 **가속도와 속도-회전 coupling** 에 반응하는 항이다.

ellipsoid의 added mass 계수는

$$\begin{aligned}\kappa_i &= \int_0^\infty \frac{r_i r_j r_k}{\sqrt{(r_i^2 + \lambda)^3 (r_j^2 + \lambda) (r_k^2 + \lambda)}} d\lambda \\ m_{A,i} &= \rho V \frac{\kappa_i}{2 - \kappa_i}\end{aligned}$$

로 주어진다. $I_{A,i}$ 도 공식 문서에 별도 식으로 정의된다.

영향:

- constant velocity에는 직접 힘을 남기지 않지만, **속도 변화에 저항한다**.
- body가 갑자기 방향을 틀 때 “즉시 따라가지 않고 묵직하게 버티는” 느낌은 added mass 성분이 핵심이다.
- current path에서 이 항은 MuJoCo ellipsoid model이 말고, Python custom added-mass term은 끈다.

4.4 Quadratic translational drag

공식 문서는 translational drag를

$$\mathbf{f}_D = -\rho \left[C_{D,\text{blunt}} A_{\mathbf{v}}^{\text{proj}} + C_{D,\text{slender}} (A_{\text{max}} - A_{\mathbf{v}}^{\text{proj}}) \right] \|\mathbf{v}\| \mathbf{v}$$

로 쓴다. 여기서 $A_{\text{max}} = 4\pi r_{\text{max}} r_{\text{min}}$ 이다.

중요한 해석:

- $C_{D,\text{blunt}}$: body를 정면으로 받았을 때의 broadside drag를 키운다.
- $C_{D,\text{slender}}$: 길쭉한 방향으로 흐를 때의 drag baseline을 키운다.
- 두 항을 섞기 때문에, 하나의 계수만으로는 안 되는 “자세에 따른 저항 차이” 를 표현할 수 있다.

4.5 Angular drag

공식 문서는 angular drag reference inertia를

$$\mathbf{I}_{D,ii} = \frac{8\pi}{15} r_i \max(r_j, r_k)^4$$

로 두고, torque를

$$\mathbf{g}_D = -\rho \boldsymbol{\omega} \left([C_{D,\text{angular}} \mathbf{I}_D + C_{D,\text{slender}} (\mathbf{I}_{\text{max}} - \mathbf{I}_D)] \cdot \boldsymbol{\omega} \right)$$

로 계산한다.

영향:

- $C_{D,\text{angular}}$ 는 roll/pitch/yaw 자체의 감쇠를 직접 키운다.
- 회전 후 계속 더 도는 tail motion을 줄이는 데 가장 직접적인 knob다.
- current scene에서 yaw가 지나치게 남는다면 fluidcoef의 세 번째 값과 geom size가 우선 검토 대상이다.

4.6 Linear viscous resistance

$$\begin{aligned} \mathbf{f}_V &= -6\pi r_D \beta \mathbf{v} \\ \mathbf{g}_V &= -8\pi r_D^3 \beta \boldsymbol{\omega} \end{aligned}$$

여기서 $r_D = (r_x + r_y + r_z)/3$ 이다.

영향:

- 저속 damping, 미세 진동, 잔류 속도 정리.
- viscosity β 가 커질수록 효과가 커진다.
- density를 안 바꾸고도 “조금 더 점성 있게” 만들 수 있다.

4.7 Magnus lift

공식 문서는 Magnus force를

$$\mathbf{f}_M = C_M \rho V (\boldsymbol{\omega} \times \mathbf{v})$$

로 근사한다.

영향:

- 회전하면서 전진할 때 lateral force가 생긴다.
- yaw rate와 surge/sway가 동시에 있을 때 **옆으로 밀리거나 기묘한 arc path** 를 만들 수 있다.
- 수중 ROV에서는 항공 곤충만큼 지배적이지 않을 수 있지만, spin-translation coupling을 설명하는 항이다.

4.8 Kutta lift

공식 문서는 Kutta condition 기반 lift를

$$\begin{aligned} \mathbf{f}_K &= \frac{C_K \rho A_{\mathbf{v}}^{\text{proj}}}{\|\mathbf{v}\|} (\mathbf{v} \times \mathbf{v}_{\parallel}) \times \mathbf{v} \\ &= C_K \rho A_{\mathbf{v}}^{\text{proj}} (\hat{\mathbf{v}} \cdot \hat{\mathbf{n}}_{s,\mathbf{v}}) (\hat{\mathbf{n}}_{s,\mathbf{v}} \times \mathbf{v}) \times \mathbf{v} \end{aligned}$$

로 제시한다.

영향:

- 받음각이나 cross-flow가 있을 때 lift-like side force를 만든다.
- body가 완전 구형이면 이 항은 0으로 간다.
- 판형/평판형 body, 혹은 특정 방향으로 납작한 body에서 더 의미가 커진다.

5 XML level에서 무엇을 바꾸면 어떤 향이 바뀌는가

XML knob	공식 모델에서 바뀌는 것	현재 v3.0에서의 실질적 의미
optiondensity	ρ 전체 스케일	drag, added mass, lift 전반 스케일 변화
optionviscosity	β 전체 스케일	저속 선형 damping, 회전 점성 저항 증가
geomfluidshape="ellipsoid"	ellipsoid model 활성화	해당 body에서 inertia model 비활성화, geom-level fluid 활성화
geomfluidcoef[0]	$C_{D,blunt}$	broadside drag 증가
geomfluidcoef[1]	$C_{D,slender}$	slender-axis drag와 angular drag 혼합 증가
geomfluidcoef[2]	$C_{D,angular}$	yaw/roll/pitch 회전 감쇠 증가
geomfluidcoef[3]	C_K	cross-flow lift 증가
geomfluidcoef[4]	C_M	spin-translation Magnus coupling 증가
geomsizes	r_x, r_y, r_z 및 projected area	어떤 축에서 저항과 added mass가 큰지 결정
geompos	fluid center 위치	같은 drag여도 CoM 기준 lever arm이 달라져 torque 체감이 달라짐

6 현재 v3.0 current scene의 실제 값

현재 active scene은 `uuv_mujoco/v2.2/scenes/tank_current_scene.xml` 이다. 여기서 fluid proxy geom은 세 개다.

geom name	position [m]	size [m]	fluidcoef
fluid_center_enclosure	(-0.005, 0.000, 0.020)	(0.205, 0.115, 0.130)	(0.620, 0.175, 1.800, 0.0, 0.0)
fluid_port_lower_body	(0.000, 0.208, 0.020)	(0.370, 0.080, 0.120)	(0.520, 0.150, 1.300, 0.0, 0.0)
fluid_starboard_lower_body	(0.000, -0.208, 0.020)	(0.370, 0.080, 0.120)	(0.520, 0.150, 1.300, 0.0, 0.0)

이 값을 읽으면 중요한 사실이 세 가지 보인다.

1. **center enclosure** 가 angular drag 계수 1.8 로 가장 크다. 즉 중앙 body가 회전 감쇠를 더 강하게 준다.
2. 좌우 lower body 두 개는 같은 계수를 가지므로, 좌우 비대칭 yaw torque를 만들기보다 **대칭적인 lateral / yaw damping scaffold** 역할을 한다.
3. C_K 와 C_M 는 현재 scene에서 **0** 이다. 즉 current v3.0 active scene은 Kutta lift와 Magnus lift를 적극적으로 쓰는 설계가 아니라, **added mass + drag + angular drag + viscous resistance 중심** 의 conservative 설정이다.

7 current profile에서 MuJoCo fluid와 함께 바뀌는 값

MuJoCo fluid model만으로 현재 응답을 다 설명할 수는 없다. current profile에는 다음 값들이 함께 들어 있다.

항목	legacy	current	현재 체감에 주는 의미
buoyancy_scale	1.003	1.008	current가 약간 더 강한 buoyancy margin을 가진다.
surface_heave_damping	18.0	0.0	legacy의 수면 근처 추가 heave damping이 current에서는 제거됐다.
cob_torque_scale	0.2	1.0	current는 CoB restoring torque를 더 직접적으로 살린다.
cob_x_offset	-0.003	0.009	fluid/부력 기준점이 legacy와 current에서 다르다.
thruster_voltage	20.0	16.0	추력 authority baseline이 current에서 더 보수적이다.
thruster_force_max	45.0	21.0	current는 thrust ceiling 자체가 더 낮다.
linear_drag	1.10	1.55	current는 저항 baseline이 더 크다.
angular_drag	0.32	0.48	current는 회전 감쇠 baseline도 더 크다.
body_inertia_scale_xyz	(1,1,1)	[1e-6, 1e-6, 1e-6]	current는 body component 합성 관성을 사실상 극단적으로 줄인다.

여기서 핵심은 current 응답 안정화가 MuJoCo fluidcoef 하나만으로 생긴 것이 아니라는 점이다. 실제 current v3.0의 안정적이고 보수적인 거동은

1. built-in ellipsoid fluid 활성화,
2. custom 6-DOF hydrodynamics 비활성화,
3. 큰 drag / angular drag baseline,
4. 매우 작은 runtime inertia scale,
5. SIM AHRS 기반 상태추정 안정화

가 겹쳐서 만들어진 결과다.

8 MuJoCo 공식 항과 현재 v3.0 코드의 대응관계

공식 항 / 기능	현재 v3.0에서 담당 주체	repository 기준 위치
ellipsoid added mass $\mathbf{f}_A, \mathbf{g}_A$	MuJoCo built-in ellipsoid fluid	tank_current_scene.xml 의 fluidshape, fluidcoef
quadratic drag $\mathbf{f}_D$	MuJoCo built-in ellipsoid fluid	same
angular drag $\mathbf{g}_D$	MuJoCo built-in ellipsoid fluid	same
linear viscous resistance $\mathbf{f}_V, \mathbf{g}_V$	MuJoCo built-in ellipsoid fluid + medium viscosity	XML density, viscosity
Magnus lift $\mathbf{f}_M$	공식 모델엔 있음, current scene에선 $C_M = 0$ 이라 사실상 비활성	fluidcoef[4]=0.0
Kutta lift $\mathbf{f}_K$	공식 모델엔 있음, current scene에선 $C_K = 0$ 이라 사실상 비활성	fluidcoef[3]=0.0
distributed buoyancy / CoB torque	Python runtime	run_urdf_full.py, sim_profiles.json
body mass / inertia synthesis	Python runtime에서 합성 후 MuJoCo body inertial 반영	sim_profiles.json body_components
thruster lag / deadzone / scaling	Python runtime	run_urdf_full.py, thruster_params.json

8.1 무엇을 MuJoCo에 맡기고 무엇을 runtime이 보정하는가

이 부분을 잘못 설명하면 “current가 built-in ellipsoid fluid model이다” 라는 말이 과장된다. 정확한 설명은 아래와 같다.

- **Fluid drag / added mass / lift 계산:** MuJoCo 공식 ellipsoid model
- **Hydrostatic buoyancy / restoring:** Python runtime
- **Thruster actuator shaping:** Python runtime
- **Rigid-body integration:** MuJoCo
- **Sensor truth export to ArduSub SITL:** Python runtime bridge

따라서 current path는 **MuJoCo fluid term**을 주력으로 쓰되, **vehicle-specific hydrostatic/actuation/interface**는 runtime에서 유지하는 하이브리드 구조다.

9 Legacy와 current의 핵심 차이

9.1 Legacy는 무엇이었는데

legacy path는 본질적으로 **Python runtime이 coefficient-based 6-DOF hydrodynamics**를 직접 계산하는 구조였다. 즉

- added mass
- added-mass Coriolis
- linear damping
- quadratic damping

을 runtime helper가 body frame에서 계산해 `xfr_c_applied` 로 넣었다.

9.2 Current는 무엇으로 바뀌었는데

current path는 이 custom hydro force를 끄고, 대신

1. scene에서 `fluidshape="ellipsoid"` 활성화
2. geom별 `fluidcoef` 설정
3. current profile로 drag / inertia / buoyancy / thruster authority 재구성

구조로 바뀌었다.

9.3 안정화에 가장 크게 기여한 것은 무엇인가

fluid force 관점만 놓고 보면 가장 크게 작용한 변화는 다음 두 개다.

1. custom 6-DOF hydrodynamic wrench를 끄고 MuJoCo built-in ellipsoid fluid를 주력으로 쓴 것
2. current profile에서 inertia scale을 극단적으로 낮추고 drag / angular drag baseline을 키운 것

첫 번째 변화는 force model을 더 shape-aware 하게 바꿨고, 두 번째 변화는 그 모델의 결과가 자세/속도에 더 빠르게 정리되도록 만들었다.

다만 **depth hold 안정화 전체**를 설명할 때는 fluid model만으로는 불충분하다. 실제 depth-hold 무한하강 문제를 끝낸 결정적 수정에는 AHRs_EKF_TYPE=10, RNGFND1_* 정렬 같은 **상태추정/센서 backend 수정**이 포함된다. 즉 **fluid model 개선과 estimator 안정화가 함께 들어가야 현재 closed-loop가 성립한다**.

10 각 항을 튜닝하면 어떤 현상이 바뀌는가

조정 항목	먼저 나타나는 현상
ρ 증가	가속 저항, drag, lift 전반 증가. 물이 더 "무거워짐"
β 증가	저속 damping 증가, 잔류 속도와 잔진동 감소
$C_{D,blunt}$ 증가	broadside drift, 측면 밀림, 수평 이동 저항 증가
$C_{D,slender}$ 증가	전진축/길이방향 저항 증가, angular drag 혼합 증가
$C_{D,angular}$ 증가	yaw/roll/pitch tail motion 감소, 회전 정리 빨라짐
C_K 증가	받음각이 있는 움직임에서 side-force / lift 커짐
C_M 증가	spin과 translation이 섞일 때 옆으로 휘는 coupling 커짐
geom size 증가	projected area, volume, added mass 모두 증가
geom pos 이동	같은 힘이어도 CoM 기준 torque가 달라짐

11 이 모델의 한계

공식 문서가 강조하듯 MuJoCo fluid model은 **state-less phenomenological model** 이다. 따라서 다음은 기대하면 안 된다.

- wake shedding
- 자유수면 wave interaction
- pressure field 직접 해석
- 두 body 사이의 복잡한 유동 간섭
- 자유표면 근처의 세밀한 공기-물 혼상 효과

즉 current path가 legacy보다 더 안정적이고 설명력 있는 것은 맞지만, 이것이 CFD 대체물이라는 뜻은 아니다. 이 모델의 장점은 실시간 제어기와 붙일 수 있을 만큼 빠르면서도, 방향성 drag/added-mass/lift를 보수적으로 표현할 수 있다는 점이다.

12 결론

MuJoCo 공식 문서 기준으로 현재 v3.0 current path를 가장 정확하게 설명하면 다음 한 문장으로 정리된다.

현재 모델은 **MuJoCo built-in ellipsoid fluid model** 을 주력 fluid force 엔진으로 사용하고, 그 주변의 **buoyancy, distributed mass/inertia, thruster shaping, SITL sensor bridge** 를 Python runtime이 보정하는 **하이브리드 수중 시뮬레이션 구조**다.

그리고 이 current path의 안정화는 단일 계수 하나에서 나온 것이 아니다. 공식 **ellipsoid fluid force**의 도입, **current profile**의 보수적 **drag / inertia** 설정, **상태추정 경로의 SIM backend 정렬**이 동시에 맞물리면서 지금의 거동이 나온다.

공식 소스 링크

- MuJoCo official docs: [Fluid forces](#)
- MuJoCo official docs: [XML reference](#)